

Section 1. Definitions of Tree Variants

Task: Write your own definitions for each tree type. You may use AI for learning, but rewrite in your own words.

1. General Tree

Definition: 最基本的樹狀結構，不限制每個節點可以擁有的子節點數量

2. Binary Tree

Definition: 規定每個節點最多只能有兩個子節點

3. Complete Binary Tree

Definition: 除了最後一層，每一層都被填滿，且最後一層的節點必須由左向右依序排列

4. Binary Search Tree (BST)

Definition: 左子節點的值小於根節點，右子節點的值大於根節點

5. AVL Tree

Definition: 嚴格自平衡的二元搜尋樹，任一節點的左右子樹高度差不超過 1。若違反規則，會透過旋轉重新平衡

6. Red-Black Tree

Definition: 另一種平衡二元搜尋樹，透過為節點標記紅色或黑色並遵循特定規則來維持大致的平衡

7. Max Heap

Definition: 完全二元樹，且滿足堆積屬性，父節點的值永遠 $\geq$ 其子節點的值

8. Min Heap

Definition: 完全二元樹，且滿足堆積屬性，父節點的值永遠 $\leq$ 其子節點的值

Section 2. Tree Family Hierarchy and Transformations

Task: Show how these structures are related (general $\rightarrow$ specialized). Use a simple diagram and explanations of what constraints are added at each step.

2.1 Tree Family Diagram

You may draw this by hand and paste a photo, or use drawing tools.

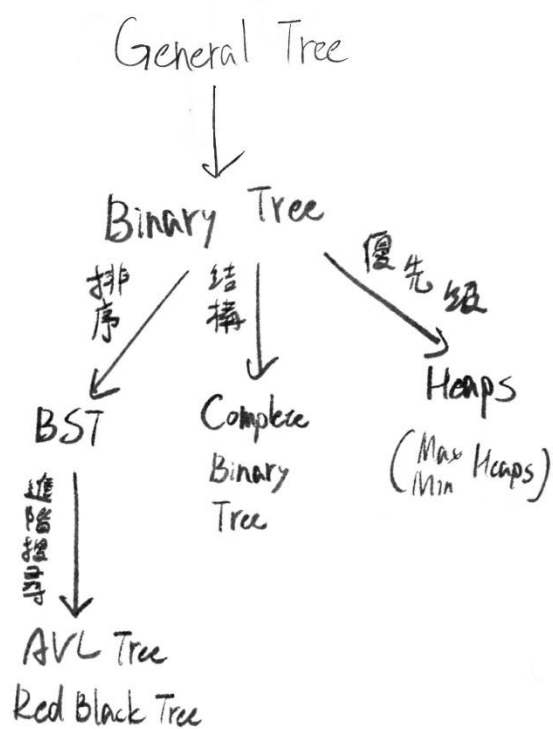
Suggested chain example (you may extend or adjust):

General Tree $\rightarrow$ Binary Tree $\rightarrow$ Complete Binary Tree

Binary Tree $\rightarrow$ Binary Search Tree $\rightarrow$ AVL / Red-Black

Binary Tree $\rightarrow$ Max Heap / Min Heap

Your Diagram:



2.2 Explanation of Transformations

Fill in what new property or constraint is added at each step.

From	To	New property / constraint added
General Tree	Binary Tree	每個節點最多只能有兩個子節點
Binary Tree	Complete Binary Tree	除了最後一層外，每一層都必須全滿，且最後一層的節點必須由左至右排列，
Binary Tree	Binary Search Tree	任一節點，其左子樹所有節點值 $<$ 根節點值 $<$ 右子樹所有節點值
BST	AVL Tree	任何節點的左右子樹高度差不超過 1
BST	Red-Black Tree	增加顏色標記與路徑平衡規則，透過節點變色與旋轉，確保從 root 到 leaf 的最長路徑不超過最短路徑的兩倍
Binary Tree	Max Heap	完全二元樹，且滿足堆積屬性。父節點的值永遠 $\geq$ 其子節點的值
Binary Tree	Min Heap	完全二元樹，且滿足堆積屬性。父節點的值永遠 $\leq$ 其子節點的值

Section 3. Tree Constructions Using Given Integers

Given integers (fixed for all parts):

37, 142, 5, 89, 63, 117, 24, 176, 58, 133, 92, 11, 151, 72, 39, 184, 7, 101, 54, 160

Task: For each tree type below, construct the tree using these integers, take a screenshot of the tree from your chosen tool, record the tool name/URL, and describe the insertion / heap-building procedure.

3.1 Binary Tree

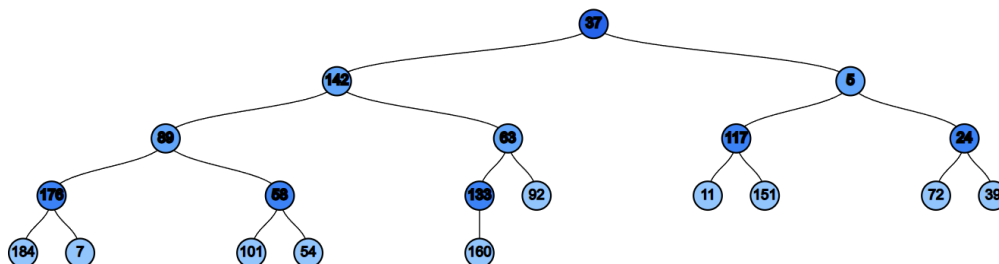
Tool name / URL:

<https://treeconverter.com/>

Construction / insertion description:

依照層級順序 (Level-order) 放入

Screenshot of Binary Tree (paste below):



3.2 Complete Binary Tree

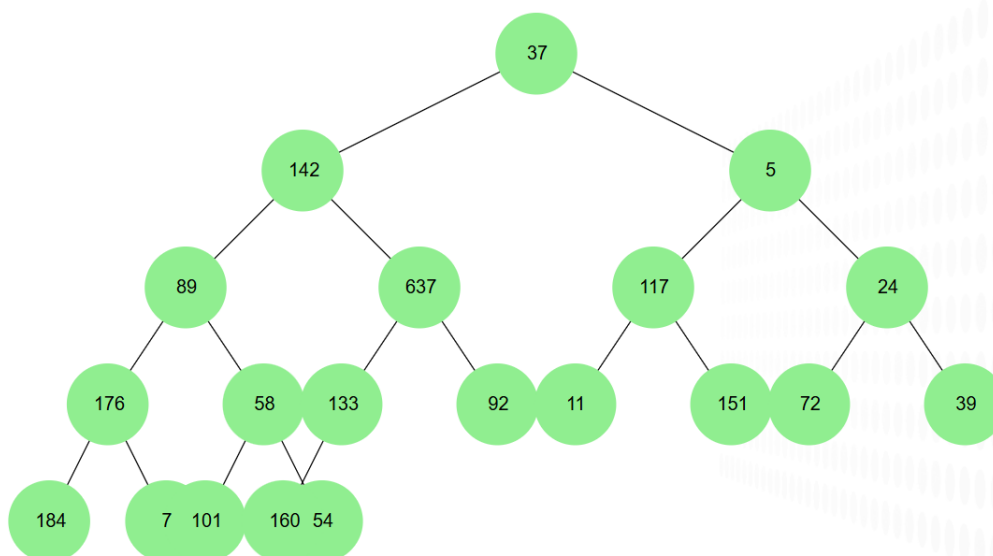
Tool name / URL:

<https://trees-visualizer.netlify.app/trees>

Construction / insertion description:

嚴格依照從上到下，從左到右的順序填滿

Screenshot of Complete Binary Tree (paste below):



3.3 Binary Search Tree (BST)

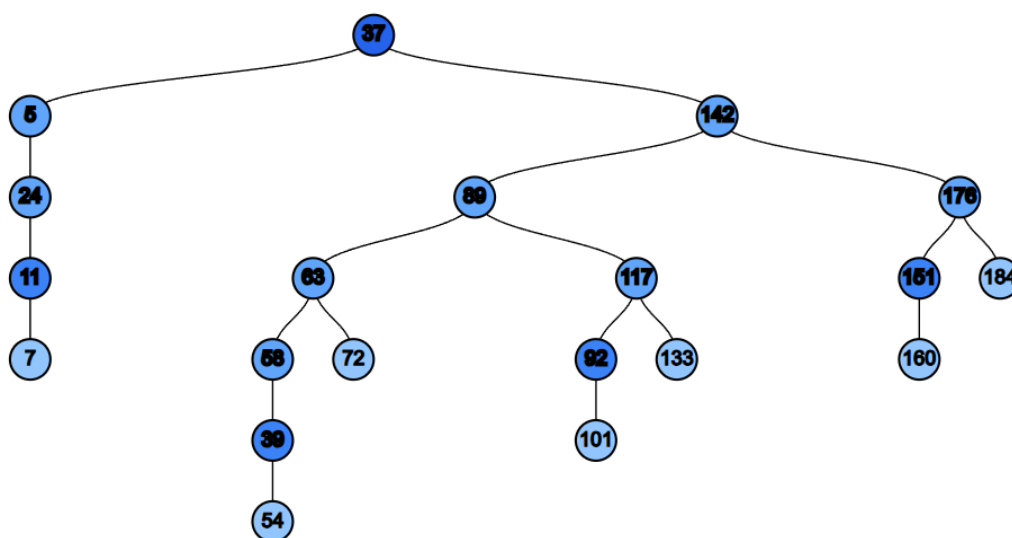
Tool name / URL:

<https://treeconverter.com/>

Insertion rule (e.g., "insert in given order using BST rules"):

以第一個數 37 為 Root，後續比 37 小放左邊，大放右邊再與後續子節點比較大小

Screenshot of BST (paste below):



3.4 AVL Tree

Tool name / URL:

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Insertion & balancing description:

插入後若失衡，需進行 LL RR LR RL 旋轉

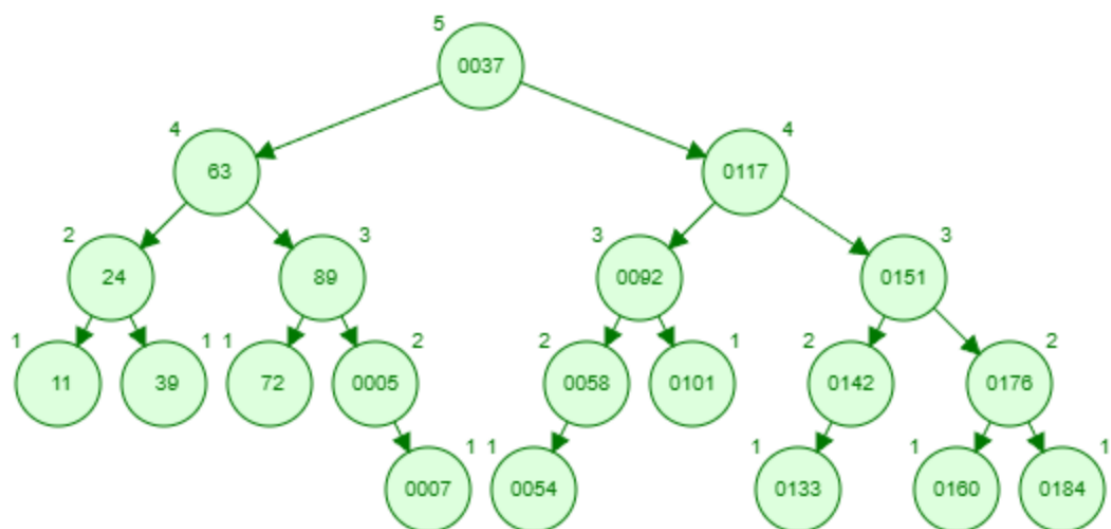
LL：將失衡節點向右旋轉

RR：將失衡節點向左旋轉

LR：先對左子節點進行左旋，再對失衡節點進行右旋

RL：先對右子節點進行右旋，再對失衡節點進行左旋

Screenshot of AVL Tree (paste below):



3.5 Red-Black Tree

Tool name / URL:

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

Insertion & balancing description:

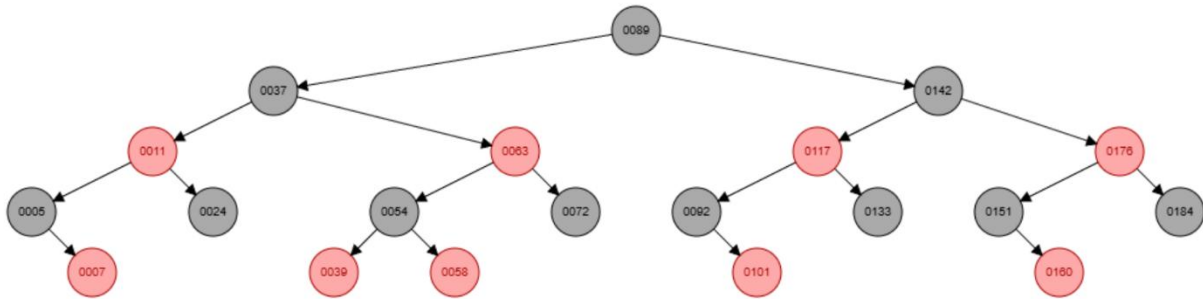
插入後透過 Recoloring 與旋轉維持平衡。

Recoloring:

當新插入節點 Z 的 Parent 是紅色，且其叔叔節點 Uncle 也是紅色時：

1. Parent 變色：將 Parent 改為黑色。
2. Uncle 變色：Uncle 改為黑色。
3. Grandparent 變色：將 Grandparent 改為紅色。
4. 向上檢查：將 Grandparent 視為新的新插入節點，重新檢查是否違反規則。
如果 Grandparent 是 Root，則必須強制改回黑色。

Screenshot of Red-Black Tree (paste below):



3.6 Max Heap

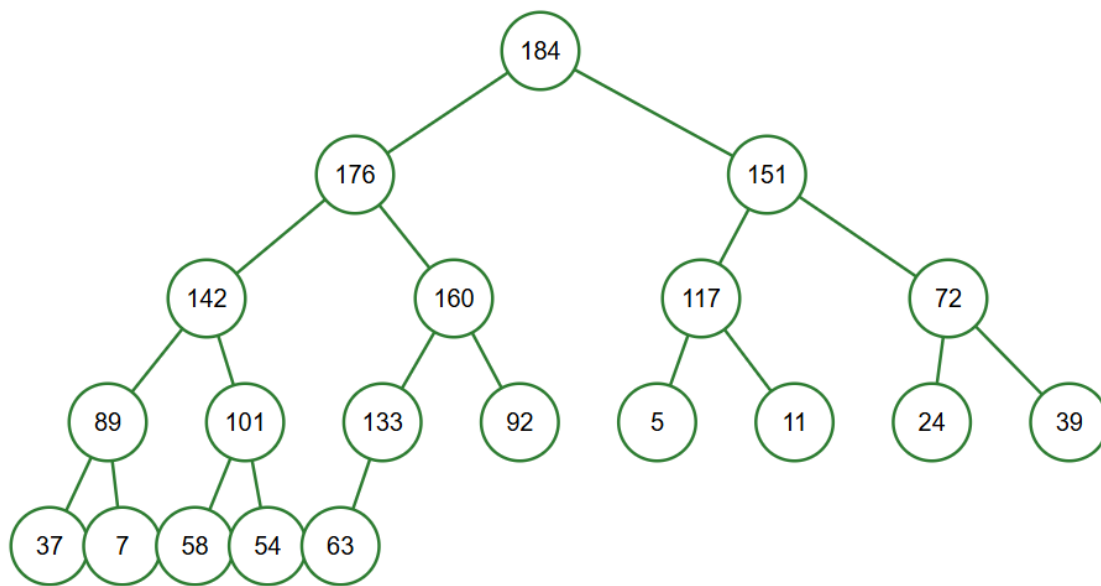
Tool name / URL:

https://sercankulcu.github.io/files/data_structures/slides/Bolum_08_Heap.html

Construction / heap-building description (e.g. heapify, insert-and-sift-up):

插入後執行 Sift-up(插在完全二元樹最後一個可用的位置後與父節點比較、交換)，確保父節點 $\geq$ 子節點

Screenshot of Max Heap (paste below):



3.7 Min Heap

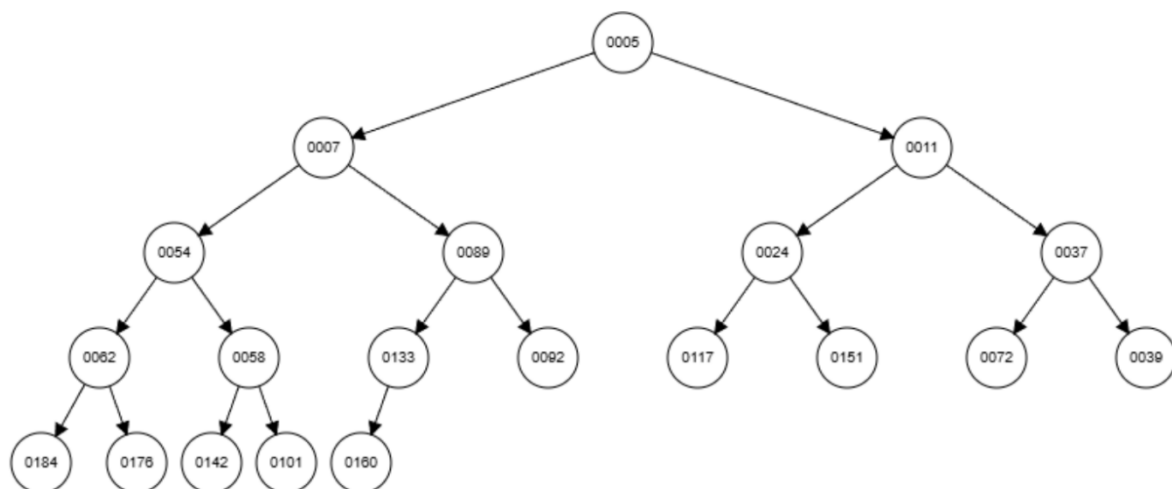
Tool name / URL:

<https://www.cs.usfca.edu/~galles/visualization/Heap.html>

Construction / heap-building description:

插入後執行 Sift-up，確保父節點 $\leq$ 子節點

Screenshot of Min Heap (paste below):



Section 4. Application Examples

Task: For each tree type, choose one application and explain why this tree is suitable.

Tree Type	Application Example (name / context)	Why this tree fits (properties that matter)
Binary Tree	Expression Trees 運算式樹	可用於表示數學運算式，其中 Leaf 是操作數，內部節點是運算子，能清楚展現運算優先順序
Complete Binary Tree	陣列式儲存 / 堆積基礎	結構完整且由左至右填滿，最適合用陣列來實作，不會浪費記憶體空間，且索引計算快速
Binary Search Tree	簡單字典或資料庫索引	具有左小右大的特性，在資料隨機分佈的情況下，能提供平均 $O(\log n)$ 的搜尋效率
AVL Tree	高度頻繁搜尋的唯讀系統	由於其高度嚴格平衡，搜尋效率非常穩定且極快，適合用於更新頻率低但查詢次數極多的場景
Red-Black Tree	作業系統排程或 C++ STL	平衡要求較 AVL 鬆散，但在插入和刪除時旋轉次數較少，能兼顧搜尋效率與修改效能
Max Heap	Priority Queue 優先權隊列	根節點始終保持最大值，這使得獲取當前最高優先權為 $O(1)$

Min Heap	Dijkstra 最短路徑演算法	根節點始終為最小值，在路徑規劃演算法中，能快速選出當前權重最小（距離最近）的下一個節點
----------	------------------	---

Section 5. Reflection on Tree Family and Performance (Optional but recommended)

Among BST, AVL, and Red-Black trees, which one would you pick for:

Mostly search (few updates)? Why?

AVL Tree

因為它最平衡，搜尋路徑最短

Frequent insertions and deletions? Why?

Red-Black Tree

因為它的平衡要求較寬鬆，調整頻率較低，效能較穩定

If you must store these 20 integers for static search only (no updates), which structure or representation would you prefer (sorted array + binary search, BST, AVL, etc.)? Why?

Sorted Array + Binary Search

因為資料量小且 no update，而陣列空間最精簡且快取友善

Section 6. AI Usage Log (Required)

Task: Record every time you ask an AI assistant about this assignment.

Index	Date / Time	AI Service (ChatGPT, Gemini, etc.)	Your Full Prompt / Question
1	12/27	Gemini	General tree Binary tree Complete binary tree Binary search tree (BST) AVL tree Red-Black tree Max heap Min heap 的定義
2	12/27	Gemini	每個數的插入規則

3	12/27	Gemini	RR LL LR RL 旋轉是甚麼
4	12/27	Gemini	紅黑樹 Recoloring 的規則
5	12/27	Gemini	Sift-up 的意思
6	12/27	Gemini	每種樹的應用場景，並解釋為何適合，且說明應用範例的運作方式